

Assignment 2

Introduction to Blockchain (ITB)

Semester: Spring 2026

Instructor: Ahmed Akhtar

Date: Thursday, 26th March 2026

Members

1. Zuha Aqib, 26106
2. Izza Sohail Khan, 28996

Table of Contents

Members.....	1
Table of Contents.....	1
Question 1 [5 marks].....	2
Question 2 [6 marks].....	3
Part (a).....	3
Part (b).....	3
Part (c).....	4
Question 3 [5 marks].....	5
Part (a).....	5
Part (b).....	5
Question 4 [5 marks].....	7
Question 5 [5 marks].....	8
Question 6 [6 marks].....	9
Part (a).....	9
Part (b).....	9
Question 7 [4 marks].....	10
Question 8 [4 marks].....	11

Question 1 [5 marks]

In Bitcoin, each user's balance is represented using the hash of a public key. A user's name or identity is not stored on the blockchain. An adversary may still use the transaction history associated with a public key in order to arrive at the identity of a user. Give an example of how this is possible and devise two strategies that may be used to protect the anonymity of a user.

Bitcoin is often described as pseudonymous rather than anonymous. While no real-world identity is stored on the blockchain, every transaction is permanently and publicly recorded against a Bitcoin address. An adversary can exploit this public ledger through a technique called transaction graph analysis to de-anonymise users.

For example, a user who purchases Bitcoin on a regulated exchange such as Coinbase must complete KYC (Know Your Customer) verification, submitting their name, address, and government-issued ID. This creates a direct, documented link between their real identity and their Bitcoin address. If that same user then sends Bitcoin to an online retailer to pay for a physical delivery, the retailer now has both the sending address and the user's shipping address. An adversary who observes this pattern (perhaps by subpoenaing the exchange or the retailer, or simply by analysing the public ledger) can now link the user's identity to their address and reconstruct their entire spending history. By following the chain of transactions, the adversary can identify counterparties, infer business or personal relationships, and build a detailed financial profile of the user. Companies like Chainalysis have commercialised exactly this kind of analysis at scale.

Strategy 1: Address Rotation

Rather than reusing a single Bitcoin address, a user should generate a fresh key pair and address for every transaction. Most modern wallets (HD wallets) do this automatically using the BIP-32 standard. This fragments the user's activity across many unlinkable addresses, making graph analysis far harder. However, the protection breaks down if the user ever consolidates funds from multiple addresses into a single transaction, since the blockchain then reveals that all those input addresses are controlled by the same entity, a heuristic known as common-input-ownership.

Strategy 2: CoinJoin Mixing

A user can route transactions through a mixing service implementing CoinJoin. Multiple users pool their inputs into a single large transaction; the outputs are shuffled and sent to new addresses, breaking the direct link between any input and its corresponding output. For instance, if three users each contribute 1 BTC and each receive 1 BTC back at a new address, an outside observer cannot determine who received what. Limitations include: some jurisdictions treat participation in mixing services as inherently suspicious and subject to regulatory scrutiny; a poorly implemented mixer may retain internal logs vulnerable to subpoenas; and amount correlation attacks remain possible if output amounts are unequal. Privacy-native cryptocurrencies such as Monero embed mixing and stealth address functionality directly into the protocol, providing stronger and more consistent privacy guarantees.

Question 2 [6 marks]

Part (a)

a) Explain why strictly more than half of the network resource guarantees eventual chain overtake.

Bitcoin's longest chain rule defines the valid chain as the one with the greatest cumulative proof of work. A chain's growth rate is directly proportional to the hash power dedicated to it. If an attacker strictly controls more than 50% of the total hash power, their chain grows faster than the honest chain in expectation. Even if the attacker starts behind, the probability that their chain eventually catches up and overtakes the honest chain converges to 1 as time goes on.

This can be understood via a random walk argument. If the attacker has fraction p of hash power and the honest network has $q = 1 - p$, the probability that an attacker who is z blocks behind eventually catches up is 1 if $p > q$, and $(p/q)^z < 1$ if $p < q$. At exactly 50%, $p = q$ and the walk is unbiased: neither chain is guaranteed to dominate, and convergence to either outcome is not assured. Only strictly greater than 50% makes eventual overtake certain.

Part (b)

b) Assume the network consensus currently considers Block#271 to be the latest block. A malicious miner controlling 51% of the network has the ability to create a private fork. Describe a double-spend attack that this miner could use against an exchange that credits deposits only after k confirmations (the exchange deposits cash into the user's account in exchange for cryptocurrency after k blocks have been mined after it). Be explicit about the miner's actions and the point at which the exchange would be tricked.

At Block 271, the malicious miner (controlling 51% of hash rate) begins secretly mining a private fork from Block 271, never broadcasting these blocks. Simultaneously, on the public chain, the miner submits a transaction sending 10 BTC to the exchange's deposit address. The honest network mines this into Block 272 and continues building. The miner waits until the deposit transaction receives k confirmations, meaning k additional blocks are mined on top of the block containing it. The exchange then treats the deposit as final, credits the miner's account with the equivalent cash value, and the miner immediately withdraws it.

Throughout this period, the miner's private chain has also been growing from Block 271. Because the miner controls 51% of hash rate, their private chain grows slightly faster than the honest chain in expectation, so by the time k public blocks have been mined, the miner's private chain is at least as long, likely longer. Crucially, this private chain does not contain the deposit transaction. Instead it contains a conflicting transaction redirecting those same 10 BTC back to a wallet the miner controls. The miner then broadcasts their longer private chain. All honest nodes, following the longest chain rule, discard the public chain and adopt the miner's chain as canonical history. The deposit transaction is erased from the ledger entirely. The exchange has no on-chain record of receiving any funds, yet the miner has already withdrawn the cash. The same coins were spent twice.

Part (c)

c) Give reasons why 51% attacks are still rare in practice.

51% attacks remain rare for several compounding reasons. First, the raw cost of acquiring 51% of Bitcoin's hash rate is prohibitive, requiring hundreds of millions of dollars in specialised ASIC hardware, and no rental market for that scale of hash power exists. Second, there is a powerful self-defeating economic incentive: a successful attack would almost certainly collapse Bitcoin's market price, destroying the value of the attacker's own hardware and any Bitcoin holdings simultaneously, making the expected profit negative. Third, exchanges have adapted by requiring large numbers of confirmations (sometimes 6 or more for high-value deposits), extending the time the attacker must sustain their private fork and multiplying the cost. Fourth, the network is actively monitored by researchers and companies, so an abnormally long private chain appearing suddenly would be flagged immediately, narrowing the window for the attacker to extract profit before the community responds. Finally, the large mining pools that collectively control most of Bitcoin's hash rate are financially invested in the network's continued legitimacy and have strong reputational and legal incentives not to attack it.

Question 3 [5 marks]

Imagine that a blockchain is about to activate a soft fork that reduces the block size limit. Most miners have upgraded, but Aqib controls 51% of the network's compute power and refuses to upgrade.

Part (a)

a) Describe what will happen to the chain if Aqib mines blocks that violate the new, smaller block size limit. Will the chain split temporarily, permanently, or not split at all?

In a soft fork, the new rules are strictly more restrictive than the old ones. Upgraded nodes apply validity checks before the longest-chain rule: a block that fails validity is discarded entirely, regardless of how much proof of work backs it or how long the chain it belongs to is. Since Aqib refuses to upgrade, every large block he produces violates the new block size limit and is therefore deemed invalid by the upgraded 49% of the network. Those nodes will not build on his chain under any circumstances and will instead continue mining their own chain of smaller, valid blocks.

Aqib's chain is valid under the old rules and grows faster due to his 51% hash power majority. However, chain length is irrelevant to upgraded nodes because validity is assessed independently of length. A longer invalid chain is still an invalid chain. Upgraded nodes will therefore never adopt Aqib's chain regardless of how much it grows, and Aqib will never adopt theirs because his software does not enforce the new rules. Since neither side will accept the other's blocks under any circumstances, the result is a permanent chain split. One chain is followed by Aqib and any remaining non-upgraded nodes; the other is followed by upgraded nodes. This is the most dangerous failure mode for a soft fork: when the miner majority refuses to comply, the upgrade cannot be enforced through hash power alone and the network fractures irreversibly..

Part (b)

b) Describe what will happen in the other scenario where Aqib, with 51% of compute power upgrades, while all the other nodes stay non-updated. Will the chain split temporarily, permanently, or not split at all? Will he be able to double-spend there?

This scenario resolves cleanly due to the backward-compatible nature of a soft fork. Because the new rules are strictly more restrictive, any block valid under the new rules is automatically valid under the old rules as well. Non-upgraded nodes will therefore accept Aqib's smaller blocks without issue, as those blocks do not violate anything in the old ruleset. When non-upgraded nodes occasionally mine a larger block, Aqib's upgraded node rejects it and continues building on his own chain. Since Aqib controls 51% of the hash rate, his chain grows faster. Non-upgraded nodes, following the longest chain rule and finding Aqib's chain perfectly valid under their old rules, naturally adopt it. The larger blocks mined by non-upgraded nodes are orphaned. The result is no permanent split; at most there are temporary short-lived forks that resolve in Aqib's favour, which is exactly the intended behaviour of a soft fork when the hash power majority has upgraded.

Regarding double-spending: Aqib's 51% hash power does give him the theoretical capacity to execute a double-spend attack, since the mechanics of mining a private fork faster than the honest network are unchanged. However, this vulnerability is a consequence of his hash power majority alone and has nothing to do with the soft fork. The soft fork itself introduces no new double-spend surface

Question 4 [5 marks]

When a soft fork is invoked, does it need the majority of nodes to upgrade to ensure that a single chain emerges? Why or why not?

A soft fork does not require a majority of nodes to upgrade for a single chain to emerge. What matters is a majority of hash power, not a majority of nodes. The reason lies in backward compatibility. Because the new rules are strictly more restrictive, any block that satisfies the new rules is automatically valid under the old rules as well. Non-upgraded nodes will never reject a block produced by upgraded miners, since those blocks do not violate the old ruleset. Non-upgraded nodes therefore follow the upgraded chain passively, without even being aware a rule change has occurred.

Chain convergence is driven by hash power. If the majority of miners upgrade, their chain grows faster than any competing chain produced by non-upgraded miners. When a non-upgraded miner occasionally produces a block violating the new rules, upgraded nodes reject it. Since the upgraded chain outpaces it, that block is orphaned. Non-upgraded nodes, following the longest chain rule, adopt the upgraded chain, and a single canonical chain emerges without requiring non-upgraded nodes to do anything.

It is worth noting that while hash power majority is sufficient for technical chain convergence, economically significant nodes (such as major exchanges, wallets, and payment processors) matter for social consensus. If these entities refuse to upgrade, they may not recognise transactions under the new rules as valid for their own purposes, which can create economic friction even without a chain split. However, this is a governance consideration rather than a technical one.

This stands in sharp contrast to a hard fork, which introduces rules incompatible with old nodes. In that case, non-upgraded nodes will reject new blocks outright, creating a permanent chain split unless near-universal coordination is achieved. The soft fork's backward compatibility is therefore one of its key practical advantages.

Question 5 [5 marks]

Read up on “Grover’s algorithm”. Will the existence of this algorithm (and similar algorithms) affect cryptographic hash functions such as SHA256? What about symmetric key encryption, such as AES-128?

Grover's algorithm is a quantum algorithm that provides a quadratic speedup for searching an unstructured space of N possibilities, reducing the number of operations needed from $O(N)$ to $O(\sqrt{N})$. This is not an exponential speedup; it weakens cryptographic systems rather than outright breaking them. However, its implications vary significantly depending on the primitive being attacked.

Impact on SHA-256

A classical brute-force preimage attack on SHA-256 requires approximately 2^{256} operations. Grover's algorithm reduces this to approximately 2^{128} operations. A 128-bit security level is still considered strong and practically unbreakable against any foreseeable adversary, classical or quantum. SHA-256 therefore remains secure in a post-quantum context, though its effective security margin is halved. The standard recommendation is to double output lengths when quantum resistance is required.

Impact on AES-128

AES-128 has a key space of 2^{128} . Grover's algorithm reduces the brute-force search to approximately 2^{64} operations. A 64-bit security level is considered insufficient for long-term protection, particularly against well-resourced state-level adversaries. The standard mitigation is to use AES-256, which retains an effective security level of approximately 2^{128} even under Grover's attack.

Broader context for Bitcoin

It is important to note that Grover's algorithm, while relevant to hash functions and symmetric encryption, is not the most pressing quantum threat to Bitcoin. That distinction belongs to Shor's algorithm, which provides an exponential speedup for solving the discrete logarithm problem. Bitcoin's transaction signatures rely on ECDSA over the secp256k1 curve, and Shor's algorithm would break this entirely, allowing an attacker with a sufficiently powerful quantum computer to derive a private key from a public key and forge signatures. This would be far more catastrophic than Grover's quadratic weakening of SHA-256. That said, practical fault-tolerant quantum computers capable of running either algorithm at cryptographically relevant scale remain likely decades away.

Question 6 [6 marks]

Part (a)

a) In simple words, explain what a soft fork and a hard fork are, focusing on what happens to nodes that do not upgrade.

A soft fork is a protocol change that makes the rules more restrictive than before. Because the new rules are a strict subset of the old ones, any block valid under the new rules is also valid under the old rules. Nodes that do not upgrade will still accept blocks produced by upgraded nodes, as they are unaware that a rule change has taken place. The network can continue operating on a single chain without requiring every node to upgrade, and non-upgraded nodes follow along passively.

A hard fork is a protocol change that introduces rules incompatible with the previous ruleset. Blocks produced under the new rules are considered invalid by non-upgraded nodes. This causes upgraded and non-upgraded nodes to follow diverging chains, resulting in a permanent split. Avoiding fragmentation requires near-universal coordination across the network: all economically significant participants must upgrade, otherwise two separate, incompatible blockchains persist indefinitely.

Part (b)

b) Give one example of a rule change for each type of fork (other than block size or mining reward) and explain why it would be a soft or a hard fork.

Soft fork example: Pay-to-Script-Hash (P2SH).

Requiring that all transactions use a specific script format (such as P2SH, introduced in BIP-16) is a soft fork. Under the old rules, a wider variety of output scripts were considered valid. Under the new rules, only scripts conforming to the P2SH template are accepted by upgraded nodes. However, any transaction that satisfies the new, stricter format is still syntactically valid under the old rules, so non-upgraded nodes accept it without issue. This is a soft fork because the new validity set is a strict subset of the old one.

Hard fork example: Adding a new mandatory transaction field.

If the protocol were changed to require every transaction to include a new mandatory field (such as a timestamp or protocol version number that old nodes do not understand), blocks containing these transactions would be rejected as malformed by non-upgraded nodes. The new transaction structure is not valid under the old rules, creating an incompatibility in both directions: upgraded nodes reject old-format transactions, and non-upgraded nodes reject new-format transactions. This is a hard fork because neither ruleset is a subset of the other, and the chains are forced to diverge permanently.

Question 7 [4 marks]

Suppose Aqib is a malicious miner who controls 51% of the network's compute power. This means he has complete control of the blocks he mines. As we have seen previously, he maliciously uses this power to double-spend his cryptocurrency. Is there any other way he can exploit his control over the mined blocks? Can he change the outputs of the fair transactions to his own address?

A miner controlling 51% of hash power has significant influence over which blocks are added to the chain and how they are constructed, but this control is structurally limited to block production and does not extend to the contents of transactions themselves. Beyond double-spending, a 51% miner can exploit their position in the following ways.

Transaction censorship

The miner can selectively exclude specific transactions from every block they mine, preventing those transactions from ever receiving confirmations. If the attacker controls the majority of blocks, a targeted transaction may be indefinitely stalled without the attacker ever needing to touch it directly.

Transaction ordering and Miner Extractable Value (MEV)

Within a block, the miner controls the order of transactions. This can be exploited to front-run transactions by inserting their own transaction ahead of a pending one they observe in the mempool, or to manipulate outcomes in smart contract contexts where transaction ordering affects the result. The profit extracted through this kind of ordering manipulation is known as Miner Extractable Value.

Selfish mining

The miner can withhold a newly found block rather than broadcasting it immediately, secretly extending their private chain. When the honest network finds its next block, the miner broadcasts their withheld block to orphan it, wasting the honest miner's work. This strategy allows the attacker to earn a disproportionate share of block rewards relative to their actual hash power.

However, a 51% miner cannot alter the outputs of existing valid transactions to redirect funds to their own address. Every Bitcoin transaction is protected by a digital signature generated with the sender's private key. If any field of the transaction (such as the recipient address or amount) is modified, the signature verification fails and all honest nodes reject the transaction as invalid. Since the attacker does not possess the sender's private key, they cannot produce a valid replacement signature. This cryptographic protection holds regardless of the attacker's hash power, which governs block selection but has no bearing on signature validity.

Question 8 [4 marks]

Explain the concept of non-repudiation in digital signatures. Why is this property critical for cryptocurrencies like Bitcoin? How does Bitcoin ensure it?

Non-repudiation is the cryptographic guarantee that a party who has signed a message cannot later plausibly deny having done so. When a sender signs a message using their private key, anyone in possession of the corresponding public key can verify that the signature is valid and that it could only have been produced by the holder of that private key. Since the private key is known exclusively to its owner, a valid signature constitutes unforgeable proof of authorship. This property is foundational to Bitcoin because the system operates without any central authority. There is no bank, court, or arbitrator to resolve disputes or reverse transactions after the fact. The entire trust model rests on cryptographic guarantees alone. If users could credibly deny having authorised a transaction, the system would be unable to enforce ownership of funds, and the integrity of value transfer would collapse entirely. Non-repudiation ensures that once a transaction is signed and broadcast, the sender is permanently and provably bound to it.

Bitcoin enforces non-repudiation through two complementary mechanisms. First, every transaction must be signed using the sender's private key via ECDSA (Elliptic Curve Digital Signature Algorithm) over the secp256k1 curve. The network verifies this signature against the corresponding public key before accepting the transaction. A transaction with a missing or invalid signature is rejected by all honest nodes. Second, accepted transactions are recorded on the blockchain, a distributed and append-only ledger replicated across thousands of nodes worldwide. The combination of cryptographic hash chaining and proof-of-work makes the ledger practically immutable, as altering a historical transaction would require redoing all subsequent proof-of-work, which is computationally infeasible. Together, the digital signature proves authorship at the moment of signing, and the blockchain preserves that proof permanently and tamper-resistantly, making denial impossible.